

DNS and domain Field Guide

DNS records, email authentication, DNSSEC and certificate fixes.

58 fixes, each one a written note with diagrams and a small Python and Node.js script that finds the problem and repairs it. 54 of 58 have a script in the public repo.

Before you run anything. Every script starts in a dry run: it reports what it would change and writes nothing. Read that output first, then set `DRY_RUN=false` when you are satisfied it has understood your data.

Scripts: github.com/allanninal/dns-fixes

Guides: allanninal.dev/dns

All 14 repos: github.com/allanninal

The fixes

1 API token missing required scope blocks DNS automation

Your ACME client, cert-manager, or CI pipeline was handed a Cloudflare API token, and it still fails every run with an authentication error. The token looks fine. It is active. But most DNS clients need to look up a zone before they can ever write to it, and a token that can only edit records, not read or list zones, fails on that very first step. The write permission was never the problem. Here is how to spot the missing scope and a small script that checks for it before the automation ever runs.

[diagnose_token_scope.py](#)

<https://www.allanninal.dev/dns/api-token-missing-scope/>

<https://github.com/allanninal/dns-fixes/tree/main/api-token-missing-scope>

2 Auto-renewal enabled but payment failed

Auto-renew is on. You checked the box years ago and never thought about it again. But the expiration date at the registry keeps getting closer, because the card on file was declined and the renewal charge never actually went through. Auto-renew being on is a setting, not a guarantee. Here is why the charge fails quietly, how to catch it before the domain lapses, and a script that watches for it on a schedule.

[auto_renewal_payment_failure.py](#)

<https://www.allanninal.dev/dns/auto-renewal-payment-failure/>

<https://github.com/allanninal/dns-fixes/tree/main/auto-renewal-payment-failure>

3 CAA record blocks certificate issuance for the intended CA

You point Let's Encrypt, DigiCert, or another certificate authority at your domain and issuance fails with a CAA error, even though the domain, DNS, and validation all look fine. A CAA record somewhere in the DNS tree is telling every CA who is allowed to issue for this name, and the CA you are actually using is not on that list. Here is how to prove it, how it usually happens, and how to fix the record so issuance goes through.

[caa_blocks_intended_ca.py](#)

<https://www.allanninal.dev/dns/caa-blocks-intended-ca/>

<https://github.com/allanninal/dns-fixes/tree/main/caa-blocks-intended-ca>

4 CAA blocks wildcard certificate issuance specifically

The base domain gets its certificate without any trouble. But every request for a wildcard, like *.example.com, fails with a CAA policy error from the same certificate authority that just issued fine a minute ago. This is one of the more confusing CAA problems because the plain-looking dig output for the base domain hides the record that is actually blocking you. Here is why an issuewild record causes this, and how to find and fix it.

[wildcard_caa_check.py](#)

<https://www.allanninal.dev/dns/caa-blocks-wildcard-issuance/>

<https://github.com/allanninal/dns-fixes/tree/main/caa-blocks-wildcard-issuance>

5 CAA lookup fails due to broken DNSSEC chain

A certificate renewal fails with a vague DNS problem looking up CAA error, but your CAA record is fine when you check it yourself. The real problem is one step earlier: your domain has DNSSEC turned on, and the chain of trust behind it is broken, so a validating resolver cannot confirm any answer at all, CAA included. Here is how to prove that, why it happens, and how to fix the chain instead of chasing a CAA record that was never wrong.

[caa_lookup_servfail_dnssec.py](#)

<https://www.allanninal.dev/dns/caa-lookup-servfail-dnssec/>

<https://github.com/allanninal/dns-fixes/tree/main/caa-lookup-servfail-dnssec>

6 CNAME at zone apex conflicts with NS or SOA

Someone pointed the bare domain, the one without "www", straight at a CNAME target. It looked fine in the dashboard. Then mail stopped arriving, or the domain would not resolve at all, or the DNS provider rejected the record outright. A CNAME on the zone apex breaks a rule that has been in DNS since 1987, and the fix is simple once you know what to look for.

[apex_cname_conflict.py](#)

<https://www.allanninal.dev/dns/cname-at-zone-apex/>

<https://github.com/allanninal/dns-fixes/tree/main/cname-at-zone-apex>

7 CNAME coexists with another record type at same name

Someone added a CNAME to a hostname that already had an A, MX, or TXT record, and the DNS host let both entries save. Now the zone quietly serves broken or unpredictable answers instead of rejecting the second record outright. Here is why the rule exists, how to find every broken name in your zone, and a script that cleans it up through the Cloudflare API.

[cname_coexistence.py](#)

<https://www.allanninal.dev/dns/cname-coexistence-violation/>

<https://github.com/allanninal/dns-fixes/tree/main/cname-coexistence-violation>

8 CNAME to A record type change fails on existing RRSet

You changed a hostname from a CNAME to a plain A record, your deploy tool said it worked, and the DNS host still rejects it or still serves the old alias. This happens because DNS will never let a CNAME and an A record share the same name, even for a single moment. Here is why the swap fails and a script that fixes it without the race.

[fix_cname_to_a.py](#)

<https://www.allanninal.dev/dns/cname-to-a-type-change-conflict/>

<https://github.com/allanninal/dns-fixes/tree/main/cname-to-a-type-change-conflict>

9 Hosting provider DNS check fails on a custom domain setup

You point your domain at GitHub Pages, Netlify, or Vercel, save the custom domain in the settings, and click check. It comes back "DNS check unsuccessful." No certificate, no traffic, just a red warning. Here is why the check fails, what records it actually wants to see, and how to set them so it turns green.

[custom_domain_dns_check.py](#)

<https://www.allanninal.dev/dns/custom-domain-dns-verification-failure/>

<https://github.com/allanninal/dns-fixes/tree/main/custom-domain-dns-verification-failure>

10 Dangling CNAME chain with a claimable intermediate hop

A subdomain's CNAME does not always point straight at a live host. Sometimes it points to another CNAME, which points to another CNAME, and only after a few hops does it reach a real server. If one of the names in the middle of that chain points at a cloud resource that has since been deleted, that hop is dangling, and most quick checks never notice because they only look at the first hop. Here is how to walk the whole chain, find the broken hop, and close it before someone else claims it.

[dangling_cname_chain.py](#)

<https://www.allanninal.dev/dns/dangling-cname-chain-intermediate-hop/>

<https://github.com/allanninal/dns-fixes/tree/main/dangling-cname-chain-intermediate-hop>

11 Dangling CNAME enables subdomain takeover

A subdomain like `promo.example.com` still has a CNAME pointing at a cloud service you closed months ago. Nobody noticed, because the record itself looks completely normal. Then a stranger signs up for that same service, claims the exact name your CNAME points at, and now they control what shows up on your subdomain. This is one of the quietest ways a company loses part of its own domain, and it is simple to find and simple to fix.

[dangling_cname_takeover.py](#)

<https://www.allanninal.dev/dns/dangling-cname-subdomain-takeover/>

<https://github.com/allanninal/dns-fixes/tree/main/dangling-cname-subdomain-takeover>

12 Required DKIM CNAME selectors not published

Microsoft 365 does not give you a DKIM key to paste into a TXT record. It gives you two hostnames and asks you to publish two CNAME records that point at them, so it can rotate its signing keys behind the scenes. If one CNAME is missing, wrong, or was created as a TXT record by mistake, Microsoft cannot finish rotating, DKIM signing breaks, and with DMARC turned up, real mail can start bouncing.

[dkim_selector_check.py](#)

<https://www.allanninal.dev/dns/dkim-cname-selector-not-published/>

<https://github.com/allanninal/dns-fixes/tree/main/dkim-cname-selector-not-published>

13 **DKIM public key stale after rotation**

You rotated your DKIM signing key. Nothing else changed. But now every outgoing message comes back dkim=fail instead of dkim=pass. The DNS record is not missing, it resolves fine, it just still holds the old public key that no longer matches what your mail server is signing with. Here is why that happens and how to roll a key over without breaking mail in the middle.

[dkim_key_rotation_check.py](#)

<https://www.allanninal.dev/dns/dkim-key-stale-after-rotation/>

<https://github.com/allanninal/dns-fixes/tree/main/dkim-key-stale-after-rotation>

14 **DKIM selector record missing or not propagated**

Your mail platform says it is signing every message with DKIM, but verifiers keep reporting none or permfail. Nothing about the signing changed. What is missing is the one TXT record that lets the other side actually read your public key. Here is why that happens and how to publish the record correctly.

[dkim_selector_check.py](#)

<https://www.allanninal.dev/dns/dkim-selector-missing/>

<https://github.com/allanninal/dns-fixes/tree/main/dkim-selector-missing>

15 **DKIM TXT record malformed or wrapped in quotes**

The DKIM record is there. Dig returns something. But every outbound message still fails signature verification. Nine times out of ten, someone copy-pasted the public key into the DNS host's web form and carried along a stray quote mark, a broken split, or a hidden space that quietly corrupts the key. Here is why that happens and how to fix it.

[dkim_txt_check.py](#)

<https://www.allanninal.dev/dns/dkim-txt-record-malformed/>

<https://github.com/allanninal/dns-fixes/tree/main/dkim-txt-record-malformed>

16 **DMARC record missing or malformed**

Most domains never publish a DMARC policy at all. On the ones that try, a stray space, a missing semicolon, or the tags in the wrong order is enough to break the record. Either way, mail receivers see no usable DMARC policy for your domain, so spoofed mail using your name gets no enforcement and you get no reports telling you it is happening. Here is why that happens and a script that checks the record and fixes it.

[dmarc_record_missing_or_malformed.py](#)

<https://www.allanninal.dev/dns/dmarc-record-missing-or-malformed/>

<https://github.com/allanninal/dns-fixes/tree/main/dmarc-record-missing-or-malformed>

17 **DMARC aggregate reports never arrive**

The DMARC record is published. The syntax looks right. Google and Microsoft say they support DMARC reporting. But weeks go by and the rua mailbox stays empty, no XML files, no dashboard data, nothing. This is one of the quietest DNS failures there is, because nothing ever errors or bounces to tell you it is broken. Here is why it happens and how to fix it.

[dmarc_rua_auth_check.py](#)

<https://www.allanninal.dev/dns/dmarc-rua-reports-not-arriving/>

<https://github.com/allanninal/dns-fixes/tree/main/dmarc-rua-reports-not-arriving>

18 **DMARC stuck at policy none indefinitely**

A quick check says the domain has DMARC. There is a record, reports are arriving, everything looks fine. But look closer at the policy tag and it still says p=none, the same as the day it was first published months or years ago. That one tag means DMARC is only watching, not stopping anything. Here is why domains get stuck there and how to move forward safely.

[dmarc_policy_check.py](#)

<https://www.allanninal.dev/dns/dmarc-stuck-at-p-none/>

<https://github.com/allanninal/dns-fixes/tree/main/dmarc-stuck-at-p-none>

19 **DNSSEC stuck pending during a domain transfer-in**

You moved a domain to a new registrar, DNSSEC was working fine before the move, and now the registrar's dashboard just says "DNSSEC pending" and will not clear, sometimes for weeks. Your DNS zone is still signed the whole time. What is actually missing is one small record at the registry that only the registrar can add. Here is why that happens and a small script that checks for it.

[check_ds_pending.py](#)

<https://www.allanninal.dev/dns/dnssec-pending-transfer-in/>

<https://github.com/allanninal/dns-fixes/tree/main/dnssec-pending-transfer-in>

20 **Domain nearing expiration without renewal action**

Every domain has a hard expiration date sitting quietly at the registry. Most of the time auto-renew handles it and nobody thinks about it again. Then one day auto-renew is off, the card on file expired months ago, or the renewal reminder emails went to an inbox nobody reads, and the domain drifts through its 30 or 60 day warning window with no action taken. If it lapses, the registry deletes it and the site, DNS, and email for that domain all go dark at the same time.

[check_domain_expiry.py](#)

<https://www.allanninal.dev/dns/domain-expiration-approaching/>

<https://github.com/allanninal/dns-fixes/tree/main/domain-expiration-approaching>

21 **DS record mismatch after a KSK rollover**

The Key Signing Key was rolled over on schedule. Everything looked fine from the zone's side. But the DS record sitting at the registrar was never updated to match the new key, or it was typed in wrong. Now every validating resolver that checks this domain sees a DS that matches nothing, and the whole domain returns SERVFAIL. Here is why that gap opens up and how to close it safely.

[ds_ksk_mismatch.py](#)

<https://www.allanninal.dev/dns/ds-record-mismatch-ksk-rollover/>

<https://github.com/allanninal/dns-fixes/tree/main/ds-record-mismatch-ksk-rollover>

22 **Duplicate or conflicting records for the same name**

One hostname, two answers that do not agree. Most often it is a CNAME record sitting next to an A record at the exact same name, or a stale A record left over from an old server sitting next to the current, correct one. Different resolvers can pick different winners, so the site works for some visitors and fails for others. Here is why it happens, how to spot it, and a script that cleans it up through the Cloudflare API.

[duplicate_conflicting_records.py](#)

<https://www.allanninal.dev/dns/duplicate-conflicting-records/>

<https://github.com/allanninal/dns-fixes/tree/main/duplicate-conflicting-records>

23 Duplicate record write rejected by provider API

A script tries to add a DNS record, and the provider API says no: the record already exists. This is not a bug in the API. It is a script that never checked what was already in the zone before it tried to write. Here is why Cloudflare rejects the call, and a small reconciler that lists first, then only creates or updates what actually needs to change.

[duplicate_record_write_conflict.py](#)

<https://www.allanninal.dev/dns/duplicate-record-write-conflict/>

<https://github.com/allanninal/dns-fixes/tree/main/duplicate-record-write-conflict>

24 Duplicate SPF TXT records on one domain

Somewhere along the way, a domain picked up a second v=spf1 TXT record. Maybe a new mail tool's setup wizard added its own instead of appending to the one already there. Maybe a migration left an old record behind. Either way, mail servers checking SPF now see two records that each claim to be the answer, and the rule for that case is not "pick the best one." It is "fail." Here is why that happens and a script that finds it and merges the two into one.

[duplicate_spf_records.py](#)

<https://www.allanninal.dev/dns/duplicate-spf-records/>

<https://github.com/allanninal/dns-fixes/tree/main/duplicate-spf-records>

25 Expired RRSIG signatures break validation

A domain that has been signed and working for months suddenly starts failing for anyone using a validating resolver. Nothing in the zone changed. No record was edited. The problem is time itself: every RRSIG signature carries an expiration date, and once that date passes, the signature is no longer trusted, even though the record underneath it is perfectly correct. Here is why that happens and how to get the zone signing again.

[expired_rrsig_signatures.py](#)

<https://www.allanninal.dev/dns/expired-rrsig-signatures/>

<https://github.com/allanninal/dns-fixes/tree/main/expired-rrsig-signatures>

26 Duplicate record set falsely flagged as conflict

Your Terraform plan or CI script refuses to add a weighted or multivalue record, calling it a duplicate, even though the zone is set up exactly the way it should be. This happens because most homemade dedup logic only looks at a record's name and type, and never looks at the field that makes weighted, latency, failover, and multivalue routing possible in the first place: SetID-identifier. Here is why the zone is fine, and a script that fixes the check instead of the DNS.

[false_positive_duplicate_detection.py](#)

<https://www.allanninal.dev/dns/false-positive-duplicate-detection/>

<https://github.com/allanninal/dns-fixes/tree/main/false-positive-duplicate-detection>

27 Hosted zone recreated with new NS values

You deleted a hosted zone and made a new one with the same name, thinking it would pick up right where the old one left off. Instead the domain stopped resolving. The new zone got a brand new, random set of nameservers, and your registrar is still holding onto the old ones. Here is why that happens, and how to point the registrar at the right servers again.

[check_recreated_zone_ns.py](#)

<https://www.allanninal.dev/dns/hosted-zone-recreated-ns-mismatch/>

<https://github.com/allanninal/dns-fixes/tree/main/hosted-zone-recreated-ns-mismatch>

28 **Leftover MX records from a decommissioned provider**

The migration to a new mail provider went fine. The new MX records are in the zone, mail is flowing, everyone moves on. Except nobody ever deleted the old provider's MX records. Now the domain has two sets of MX records pointing at two different mail systems, and mail servers around the internet are free to pick either one. Some of your mail may be quietly landing in a mailbox nobody has opened in months. Here is why that happens and a script that finds the leftovers and removes them.

[leftover_mx_records_after_migration.py](#)

<https://www.allanninal.dev/dns/leftover-mx-records-after-migration/>

<https://github.com/allanninal/dns-fixes/tree/main/leftover-mx-records-after-migration>

29 **Missing A or AAAA record causes NXDOMAIN**

You visit a subdomain, or your app tries to reach it, and nothing answers. Not a timeout, not a wrong page, just NXDOMAIN, the DNS way of saying "this name does not exist." Here is why that happens, how to tell it apart from a similar-looking but different problem, and how to add the missing record so the name resolves again.

[missing_address_record.py](#)

<https://www.allanninal.dev/dns/missing-address-record-nxdomain/>

<https://github.com/allanninal/dns-fixes/tree/main/missing-address-record-nxdomain>

30 **Missing DS record breaks DNSSEC validation**

You turned on DNSSEC at your DNS host. It is signing every answer. Then some resolvers stop returning your site at all, and instead just say SERVFAIL. Nothing is wrong with the signatures themselves. The missing piece sits one level up, at the registrar, where a small record called the DS record was supposed to be published and never was. Here is why that breaks everything and the exact steps to fix it.

<https://www.allanninal.dev/dns/missing-ds-record/>

31 **Missing reverse DNS for outbound mail IP**

Your mail server sends a message, the connection looks fine, and then the receiving side rejects it or drops it straight into spam. No typo in the address, no bad password, nothing obviously wrong. The real problem is one level below your domain: the IP address your mail server sends from has no reverse DNS record pointing back to a real hostname, or that record does not match. Here is why that alone is enough to get you rejected, and the exact steps to fix it.

<https://www.allanninal.dev/dns/missing-ptr-for-mail-ip/>

32 **Subdomain delegation missing NS glue in the parent zone**

The child zone looks fine. It has a real SOA record, real NS records, and it answers correctly when you ask its own nameservers directly. But the subdomain still will not resolve for anyone else. The reason is almost always the same: the parent zone never got the NS records that tell the rest of the internet where to find the child. Here is how to spot that gap and close it.

[check_delegation.py](#)

<https://www.allanninal.dev/dns/missing-subdomain-delegation-glue/>

<https://github.com/allanninal/dns-fixes/tree/main/missing-subdomain-delegation-glue>

33 **MX record points to an unreachable host**

The domain's MX record still lists a mail host, and DNS still hands back an answer for it, so everything looks configured. But that host is gone. Maybe the mail provider was migrated away from, or the old server was shut down, and nobody updated DNS to match. Mail keeps queuing quietly for a day or two, then bounces, and nobody notices until the bounce shows up. Here is why that happens and a script that finds every dead MX host and repoints the record at a live one.

[mx_points_to_dead_host.py](#)

<https://www.allanninal.dev/dns/mx-points-to-dead-host/>

<https://github.com/allanninal/dns-fixes/tree/main/mx-points-to-dead-host>

34 **MX record malformed with stray characters or a missing FQDN**

Someone typed or scripted an MX record and it did not come out right. The target has an @ sign in it, maybe pasted in from an email address by mistake, or it is a raw IP address instead of a hostname. It looks close enough to correct that nobody notices at a glance, but mail servers read it literally, and a value like that is not a real hostname. Inbound mail starts bouncing or just goes quiet. Here is why that happens and a script that finds it and fixes it.

[mx_record_malformed_target.py](#)

<https://www.allanninal.dev/dns/mx-record-malformed-target/>

<https://github.com/allanninal/dns-fixes/tree/main/mx-record-malformed-target>

35 **MX record has no A or AAAA record**

The MX record for a domain looks fine. It names a mail server, it has a preference number, it has been there for months. But mail is bouncing or queuing forever. The reason is often simple and easy to miss: the hostname the MX record points at has no A or AAAA record of its own, so every sending mail server has a name but nowhere to connect it to. Here is why that happens and how to add the missing record.

[mx_target_missing_address_record.py](#)

<https://www.allanninal.dev/dns/mx-target-missing-address-record/>

<https://github.com/allanninal/dns-fixes/tree/main/mx-target-missing-address-record>

36 **Orphaned DS record left after disabling DNSSEC**

Someone turned DNSSEC off. The zone stopped signing. And now the whole domain is down for anyone using a resolver that checks DNSSEC, like Google's 8.8.8.8 or Cloudflare's 1.1.1.1. Nothing about the zone itself is broken. The leftover piece is one small record still sitting at the registrar, one level above the zone, still telling the world "this domain is signed." Here is why that one forgotten record takes down the whole domain, and the safe order to clean it up.

<https://www.allanninal.dev/dns/orphaned-ds-after-dnssec-disabled/>

37 **Orphaned DNS records left after service teardown**

A team spins up app.example.com pointing at a load balancer or a SaaS app. Months later the underlying resource is torn down, but nobody deletes the DNS record. It sits there, quietly resolving to a name that no longer belongs to anyone, until a stranger notices and claims it first. Here is why the record outlives the service, and a script that finds every one of these before someone else does.

[orphaned_records.py](#)

<https://www.allanninal.dev/dns/orphaned-records-after-teardown/>

<https://github.com/allanninal/dns-fixes/tree/main/orphaned-records-after-teardown>

38 **Partial nameserver cutover serves split answers**

You moved your domain to a new DNS host. Some people now see the new website and new email settings. Other people, on a different network, still see the old ones. Nobody changed anything twice, yet the same domain name gives two different answers at the same time. Here is why that happens during a nameserver switch, and a small script that checks every nameserver for your domain and tells you exactly where the split is.

[check_split_answers.py](#)

<https://www.allanninal.dev/dns/partial-nameserver-cutover-split-answers/>

<https://github.com/allanninal/dns-fixes/tree/main/partial-nameserver-cutover-split-answers>

39 **Proxied record silently overrides configured TTL**

Your DNS-as-code file says ttl: 300. Your script or the dashboard form sends that value to Cloudflare. The apply step succeeds with no error. Then you check the live zone and the record shows ttl=1, not 300. This is not a bug in your tool and it is not a caching artifact. Cloudflare only honors a custom TTL on records that are not proxied, and it never tells you it ignored the number you sent. Here is why that happens and a script that catches the mismatch before it confuses your next deploy.

[proxied_record_forces_ttl.py](#)

<https://www.allanninal.dev/dns/proxied-record-forces-ttl/>

<https://github.com/allanninal/dns-fixes/tree/main/proxied-record-forces-ttl>

40 **WHOIS or RDAP change signals a possible domain hijack**

A domain does not usually get stolen in one dramatic move. It gets stolen quietly first: an attacker gets into the registrar account or the registrant's email, then edits a few fields in the domain's RDAP record. The transfer lock disappears. The nameservers change. Nobody is watching, so nobody notices until the domain, or its traffic, is already gone. Here is what that quiet change looks like and how to catch it while there is still time to act.

[check_rdap_hijack_signal.py](#)

<https://www.allanninal.dev/dns/rdap-hijack-signal-monitoring/>

<https://github.com/allanninal/dns-fixes/tree/main/rdap-hijack-signal-monitoring>

41 **Registrar nameservers do not match the hosted zone**

You moved the zone to a new DNS host, the records look right, but the host's dashboard still says the zone is pending or inactive. The reason is almost always the same: the registrar is still telling the world to use the old nameservers. Here is why the two sides can disagree, how to check both of them, and how to make the registrar match the zone.

[check_ns_mismatch.py](#)

<https://www.allanninal.dev/dns/registrar-zone-nameserver-mismatch/>

<https://github.com/allanninal/dns-fixes/tree/main/registrar-zone-nameserver-mismatch>

42 **Answers differ across public resolvers**

You changed a record, then checked it on a few public resolvers, and they do not agree. 1.1.1.1 shows the new IP address. 8.8.8.8 still shows the old one. Nobody broke anything. Each resolver is just holding onto its own cached copy of the answer for a different amount of time. Here is why that happens, how to tell it apart from a real mistake, and how to make the next change settle faster.

[check_resolver_consistency.py](#)

<https://www.allanninal.dev/dns/resolver-answer-inconsistency/>

<https://github.com/allanninal/dns-fixes/tree/main/resolver-answer-inconsistency>

43 **SPF record has a duplicate or misplaced all mechanism**

Two email vendors, two SPF snippets, one merged record, and now there are two "all" tokens sitting in it, or the "all" landed somewhere in the middle instead of at the end. The record looks fine at a glance. But SPF reads left to right and stops at the first match, so whichever "all" comes first quietly becomes your entire policy, and everything typed after it is never even looked at.

[spf_all_mechanism.py](#)

<https://www.allanninal.dev/dns/spf-duplicate-all-mechanism/>

<https://github.com/allanninal/dns-fixes/tree/main/spf-duplicate-all-mechanism>

44 **SPF exceeds the 10 DNS lookup limit**

You added Google Workspace, then Microsoft 365, then SendGrid, then Mailchimp, then HubSpot, one include at a time, and each one seemed harmless on its own. Now mail that should pass SPF is failing, and nobody changed the sending server. The record itself looks fine to read. The problem is buried in how many DNS lookups it actually costs once every include is followed all the way down, and that number just walked past 10.

[spf_exceeds_lookup_limit.py](#)

<https://www.allanninal.dev/dns/spf-exceeds-lookup-limit/>

<https://github.com/allanninal/dns-fixes/tree/main/spf-exceeds-lookup-limit>

45 **Stale ACME challenge TXT record blocks renewal**

A certificate renewal fails with an error like "incorrect TXT record" or "domain: unauthorized," even though your ACME client just published a fresh challenge. The usual cause is not a broken renewal at all. It is an old _acme-challenge TXT record from a prior failed attempt that never got cleaned up, still sitting in the zone, still a valid answer to anyone who asks. Here is why that happens and a script that finds the leftover and removes it.

[stale_acme_challenge_record.py](#)

<https://www.allanninal.dev/dns/stale-acme-challenge-record/>

<https://github.com/allanninal/dns-fixes/tree/main/stale-acme-challenge-record>

46 **Stale DS records left behind after a key rollover**

A DNSSEC key rollover finished weeks ago. The new key works fine on its own. But someone forgot one small step at the registrar: delete the old DS record. Now resolvers see two DS entries, one that matches nothing in the zone, and validation quietly starts failing for anyone who checks DNSSEC. Here is why that old record keeps causing trouble and how to clean it up safely.

[stale_ds_records.py](#)

<https://www.allanninal.dev/dns/stale-ds-records-orphaned/>

<https://github.com/allanninal/dns-fixes/tree/main/stale-ds-records-orphaned>

47 **Stale negative cache keeps a fixed record unresolvable**

You added the missing record. You checked it yourself and it works. Then a coworker, or a customer, or a monitor somewhere still gets NXDOMAIN like nothing changed. Nothing is broken. A resolver asked about the name before the record existed, was told "no such name," and is still repeating that answer because it has not been told otherwise yet. Here is why that happens and what to do about it.

[stale_negative_cache.py](#)

<https://www.allanninal.dev/dns/stale-negative-cache-nxdomain/>

<https://github.com/allanninal/dns-fixes/tree/main/stale-negative-cache-nxdomain>

48 **TLS certificate nearing or past expiry**

Let's Encrypt certificates only last ninety days, so every host quietly depends on a renewal job running on time. Most of the time it does. Then one day it does not, nobody notices, and the certificate just keeps serving traffic on borrowed time until the moment it expires and every browser and API client starts failing at once, with no warning to whoever is on call.

[check_tls_expiry.py](#)

<https://www.allanninal.dev/dns/tls-certificate-expiring/>

<https://github.com/allanninal/dns-fixes/tree/main/tls-certificate-expiring>

49 **TLS certificate SAN or hostname mismatch**

A visitor opens api.example.com and the browser stops them cold with a certificate warning, even though example.com and www.example.com load fine. The certificate is real and not expired. It just was never told about this hostname. Here is why that happens and how to reissue a certificate that actually covers every name you serve.

[tls_san_hostname_mismatch.py](#)

<https://www.allanninal.dev/dns/tls-san-hostname-mismatch/>

<https://github.com/allanninal/dns-fixes/tree/main/tls-san-hostname-mismatch>

50 **Transfer blocked by registrar lock near expiry**

The domain is about to expire, you are trying to move it to a new registrar, and the transfer keeps getting rejected. The domain is not dead yet. The registrar is still holding a lock on it, and that lock does not care how many days are left on the clock. Here is why the lock and the countdown are two separate problems, and what order to fix them in.

[check_transfer_lock_risk.py](#)

<https://www.allanninal.dev/dns/transfer-lock-blocking-renewal/>

<https://github.com/allanninal/dns-fixes/tree/main/transfer-lock-blocking-renewal>

51 **Transfer lock unexpectedly removed**

You check your domain one day and the transfer lock is just gone. Nobody on your team asked for it. Nobody clicked "unlock." But the status that used to sit there, quietly stopping anyone from moving your domain to another registrar, is no longer in the record. This one is not a DNS record problem. It is a security problem, and it needs to be treated like one right away.

<https://www.allanninal.dev/dns/transfer-lock-unexpectedly-removed/>

52 **TTL set too high delays urgent changes**

You need a DNS record fixed right now. Maybe a server moved, maybe a mail host changed, maybe something is actively broken. You update the record at your DNS host in seconds. Then nothing happens for hours. Visitors, and even you on a different network, keep getting the old, wrong answer. The record was never wrong at the DNS host. Its TTL was just set too high, so every resolver that already asked is allowed to keep the stale answer for a long time. Here is why that happens and a script that finds records with a dangerously high TTL before they cause this exact problem.

[ttl_too_high.py](#)

<https://www.allanninal.dev/dns/ttl-too-high/>

<https://github.com/allanninal/dns-fixes/tree/main/ttl-too-high>

53 **TTL set too low overloads authoritative nameservers**

Someone dropped the TTL to 30 or 60 seconds before a migration so a change would take effect fast, and it worked. Then the migration finished, everyone moved on, and nobody ever set it back. Months later the domain is getting hit with far more DNS queries than it should, nameserver response times are creeping up, and on a busy day it starts to look a lot like an attack. It is not an attack. It is a TTL that never got raised back to normal.

[ttl_too_low.py](#)

<https://www.allanninal.dev/dns/ttl-too-low/>

<https://github.com/allanninal/dns-fixes/tree/main/ttl-too-low>

54 **DNS-managed record overwritten without an ownership check**

A record that a person created by hand, or that another team's automation owns, gets quietly changed or deleted by a tool like ExternalDNS or a Terraform provider. Nobody asked for the change. Nobody reassigned the record on purpose. The reconciler just diffed its desired state against the live zone and wrote over whatever did not match, because it never checked who was actually allowed to touch that name first.

[check_record_ownership.py](#)

<https://www.allanninal.dev/dns/unowned-record-overwritten/>

<https://github.com/allanninal/dns-fixes/tree/main/unowned-record-overwritten>

55 **Wildcard record creates unintended catch-all exposure**

Someone added a wildcard record to make life easier, and now every made-up subdomain under your domain quietly resolves. Typos, old subdomains nobody remembers, and names an attacker guesses like admin or vpn all answer successfully instead of failing. Here is how to prove a wildcard is a true catch-all, why it happens, and how to narrow it down without breaking the one use case it was meant for.

[wildcard_catch_all.py](#)

<https://www.allanninal.dev/dns/wildcard-catch-all-exposure/>

<https://github.com/allanninal/dns-fixes/tree/main/wildcard-catch-all-exposure>

56 **Wildcard record points at a deprovisioned service**

Years ago someone added a wildcard record so every subdomain of your zone would route to a hosted app. That app is long gone now, closed or deleted, but the wildcard record never got cleaned up. It still answers for any name you can type, and that is exactly what a stranger needs to serve their own page on your domain with a certificate that looks trusted.

[dangling_wildcard.py](#)

<https://www.allanninal.dev/dns/wildcard-pointing-at-deprovisioned-service/>

<https://github.com/allanninal/dns-fixes/tree/main/wildcard-pointing-at-deprovisioned-service>

57 **www and apex configured inconsistently**

Someone opens the site at www.example.com and it loads fine. Someone else, or a script, or an old link, hits example.com without the www and gets a blank error, a parked page, or a completely different site. Both names are supposed to serve the same thing. When only one of them was ever set up, or the two were pointed at different places, the two names quietly drift apart and only some of your visitors notice.

[www_apex_mismatch.py](#)

<https://www.allanninal.dev/dns/www-apex-mismatch/>

<https://github.com/allanninal/dns-fixes/tree/main/www-apex-mismatch>

58 **Zone lookup fails to resolve the correct managed zone**

Your certificate request fails with an error like "could not determine the zone" or "NXDOMAIN looking up TXT record." Nothing about your certificate config looks wrong. The real problem is one level down: the ACME client walked up your domain name looking for the right DNS zone, landed on an apex, and that apex is not the exact zone name your DNS provider account has on file. Here is why that walk goes wrong, and how to make the two agree.

[resolve_acme_zone.py](#)

<https://www.allanninal.dev/dns/zone-lookup-failure-for-subdomain/>

<https://github.com/allanninal/dns-fixes/tree/main/zone-lookup-failure-for-subdomain>
